Cancer_Epigenetic 2025

Introduction to bash script (3)

Huyha

Nov 30 2025

Email: hagiahuy311@gmail.com

Contents

Classwork

Input 3 number

+Calculate **Sum**,**Mean**

+Detect which is the biggest(optional)

+Detect if the biggest (or 3 number input) is **odd** or **even**

Result

```
baschscript > $ practice1.sh
1   #!/bin/bash
2
3   sum=$(($1+$2+$3))
4   mean=$((sum/$#))
5   echo "Sum: $sum"
6   echo "mean $mean"
```

```
7   bg=$1
8   if [[ $2 -gt $bg  &&  $2 -gt $3 ]]
9   then
10      bg=$2
11      echo "$bg is the biggest"
12  elif [[ $3 -gt $bg ]]
13  then
14      bg=$3
15      echo "$bg is the biggest"
16  else
17      echo "$bg is the biggest"
18  fi
19
```

```
19
20  if [[ $(($bg%2)) -eq 0 ]];then
21  echo "$bg is even"
22  else
23  echo "$bg is odd"
24  fi
25
```

```
./practice1.sh 100 200 303
```

```
Sum: 603
mean 201
303 is the biggest
303 is odd number
```

Classwork

Input n number

+Calculate **Sum**,**Mean**

+Detect which is the biggest(optional)

+Detect if the biggest (or n number input) is **odd** or **even**

Practice 2

Write a script that prints the numbers from 15 to 115 (distance between 2 number is 15). Additionally:

1. **Highlight reaching 60:** When the script encounters the number 60, print a message indicating it reached 60.
2. **Identify odd/even:** For each number, determine if it's odd or even and print that information alongside the number.(optional)
3. **Divisibility by 10:** Indicate if the number is divisible by 10 (a multiple of 10)(optional)

```
15 is odd and indivisible for 10
30 is even and divisible for 10
45 is odd and indivisible for 10
60 is even and divisible for 10
number is reach 60
75 is odd and indivisible for 10
90 is even and divisible for 10
105 is odd and indivisible for 10
```

Homework 1

1. Create file test1.txt and test2 directory in /home/user (~)
2. Write script to find if the files in /home/user is directory or file or other type.

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ ls ~
a.out  Desktop  Documents  Downloads  homework  huyha  miniconda3  Music  Pictures  Public  R  snap  Templates  test2  Videos
```

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ ./homework2.sh
a.out is file
Desktop is directory
Documents is directory
Downloads is directory
homework is directory
huyha is directory
miniconda3 is directory
Music is directory
Pictures is directory
Public is directory
R is directory
snap is directory
Templates is directory
test2 is directory
Videos is directory
```

What awk can do?

Porcine circovirus 2 isolate PCV2 P1 OK/USA, complete genome

GenBank: MW051676.1

FASTA Graphics

Go to:

```
LOCUS       MW051676               1761 bp    DNA     circular VRL 21-JUL-2021
DEFINITION  Porcine circovirus 2 isolate PCV2 P1 OK/USA, complete genome.
ACCESSION   MW051676
VERSION     MW051676.1
KEYWORDS    .
SOURCE      Porcine circovirus 2
  ORGANISM  Porcine circovirus 2
            Viruses; Monodnaviria; Shotokuvirae; Cressdnaviricota;
            Arfiviricetes; Cirlivirales; Circoviridae; Circovirus; Circovirus
            porcine2.
REFERENCE   1  (bases 1 to 1761)
  AUTHORS   Paim,W.P., Maggioli,M.F., Weber,M.N., Rezabek,G., Narayanan,S.,
            Ramachandran,A., Canal,C.W. and Bauermann,F.V.
  TITLE     Virome characterization in serum of healthy show pigs raised in
            Oklahoma demonstrated great diversity of ssDNA viruses
  JOURNAL   Virology 556, 87-95 (2021)
   PUBMED   33550118
REFERENCE   2  (bases 1 to 1761)
  AUTHORS   Paim,W.P., Maggioli,M.F., Weber,M.N., Rezabek,G., Narayanan,S.S.,
            Ramachandran,A., Canal,C.W. and Bauermann,F.V.
  TITLE     Direct Submission
  JOURNAL   Submitted (27-SEP-2020) Veterinary Pathobiology, Oklahoma State
            University, 250 McElroy Hall, Stillwater, OK 74074, USA
COMMENT     ##Assembly-Data-START##
            Assembly Method       :: Geneious Prime v. 2020.2.1
            Assembly Name         :: PCV2 P1 OK/USA
            Coverage              :: 12.5X
            Sequencing Technology :: Illumina
            ##Assembly-Data-END##
FEATURES             Location/Qualifiers
     source          1..1761
                     /organism="Porcine circovirus 2"
                     /mol_type="genomic DNA"
                     /isolate="PCV2 P1 OK/USA"
                     /host="swine"
                     /db_xref="taxon:85708"
                     /country="USA"
                     /collection_date="2018"
```

Customize view

Analyze this sec
Run BLAST
Pick Primers
Highlight Sequenc
Find in this Sequer

Related informa
Protein
PubMed
Taxonomy
RefSeq Genome f
RefSeq Genome S

Recent activity
Porcine circov OK/USA, com
pcv2 (12756)
Porcine circov genome
pcv2 complete

```
                     GWLPWDDLLRLCDRYPLTVETKGGTVPFLARSILITSNQTPLEWYSSTAVPAVEALYR
                     RITSLVFWKNATEQSTEEGGQFVTLSPPCPEFPYEINY"
     CDS             complement(351..665)
                     /codon_start=1
                     /product="ORF3 protein"
                     /protein_id="QXO84985.1"
                     /translation="MVTIPPLVSRWFPVCGFRVCKISSPFAFATPRWPHNDVYIRLPI
                     TLLHFPAHFQKFSQPAEISDKRYRVLLCNGHQTPALQQGTHSSREVTPLSLRSRSSTF
                     YQ"
     CDS             complement(1024..1728)
                     /codon_start=1
                     /product="capsid protein"
                     /protein_id="QXO84986.1"
                     /translation="MTYPRRRFRRRRHRPRSHLGQILRRRPWLVHPRHRYRWRRKNGI
                     FNTRLSRTIGYTVKKTTVRTPSWNVDMMRFNINDFLPPGGGSNPLTVPFEYYRIRKVK
                     VEFWPCSPITQGDRGVGSTAVILDDNFVTKANALTYDPYVNYSSRHTITQPFSYHSRY
                     FTPKPVLDRTIDYFQPNNKRNQLWLRLQTTGNVDHVGLGTAFENSIYDQDYNIRITMY
                     VQFREFNLKDPPLNPN"
ORIGIN
        1 gcacttcggc agcggcagca cctcggcagc acctcagcag caacatgccc agcaagaaga
       61 gtggaagaag cggaccccaa ccacataaaa ggtgggtgtt cacgctgaat aatccttccg
      121 aagacgagcg caagaaaata cgggagctcc caatctccct atttgattat tttattgttg
      181 gcgaggaagg taatgaggag ggccgaacac cccacctaca ggggttcgct aattttgtga
      241 agaagcaaac ttttaataaa gtgaagtggt attttggtgc ccgctgccac atcgagaaag
      301 cgaaaggaac agatcagcag aataaagaat attgcagtaa agaaggcaac ttactgatag
      361 aatgtggagc tcctagatct caaggacaac ggagtgacct ctctactgct gtgagtacct
      421 tgttggagag cgggagtctg gtgaccgttg cagagcagca ccctgtaacg tttgtcagaa
      481 atttccgcgg gctggctgaa cttttgaaag tgagcgggaa aatgcagaag cgtgattgga
      541 agacgaatgt acacgtcatt gtggggccac ctgggtgtgg caaaagcaaa tgggctgcta
      601 attttgcaga cccggaaacc acatactgga aaccacctag aaacaagtgg tgggatggtt
      661 accatggtga agaagtggtt gttattgatg acttttatgg ctggctgccg tgggatgatc
      721 tactgagact ctgtgatcga tatcctttga ctgttgagac taaaggtgga actgtacctt
      781 ttttggcccg cagtattctg attaccagca atcagacccc gttggaatgg tactcctcaa
      841 ctgctgtccc agctgtagaa gctctctatc ggaggattac ttccttggta ttttggaaga
      901 atgctacaga acaatccacg gaggaagggg gccagttcgt caccctttcc cccccatgcc
      961 ctgaatttcc atatgaaata aattactgag tctttttttat cacttcgtaa tggttttat
     1021 tattcagtta gggttaagtg gggggtcttt aagattaaat tctctgaatt gtacatacat
     1081 ggttatacgg atattgtagt cctggtcgta tatactgttt tcgaacgcag tgccgaggcc
     1141 tacatggtct acatttccag tagtttgtag tctcagccag agttgatttc ttttgttatt
     1201 gggttggaag taatcgattg tcctatcaag gacaggtttc ggggtaaagt accgggagtg
     1261 gtaggagaag ggctgggtta tggtatggcg ggaggagtag tttacatagg ggtcataggt
     1321 tagggcattg gcctttgtta caaagttatc atctagaata acagcagtgg agcccactcc
     1381 cctgtcaccc tgggtgattg gggagcaggg ccagaattca accttaacct tccttattct
     1441 gtagtattca aagggcacag tgagggggtt tgagccccct cctgggggaa gaaaatcatt
     1501 aatattaaat ctcatcatgt ccacattcca ggagggcgtt ctgactgtgg ttttcttgac
     1561 agtgtaaccg atggtgcggg agaggcgggt gttgaagatg ccatttttcc ttctccagcg
     1621 gtaacggtgg cgggggtgga cgagccaggg ccggcggcgg aggatctggc caagatggct
     1681 gcgggggcgg tgtcttcgtc tgcggaaacg cctccttgga tacgtcatcg ctgaaaacga
     1741 aagaagtgcg ctgtaagtat t
//
```

11.

Porcine circov

GenBank: MW051676.1

FASTA Graphics

Go to:

```
LOCUS       MW05167
DEFINITION  Porcine
ACCESSION   MW05167
VERSION     MW05167
KEYWORDS    .
SOURCE      Porcine
  ORGANISM  Porcine
            Viruses
            Arfivir
            porcine
REFERENCE   1  (bas
  AUTHORS   Paim,W.
            Ramacha
  TITLE     Virome
            Oklahom
  JOURNAL   Virolog
   PUBMED   3355011
REFERENCE   2  (bas
  AUTHORS   Paim,W.
            Ramacha
  TITLE     Direct
  JOURNAL   Submitt
            Univers
COMMENT     ##Assem
            Assembl
            Assembl
            Coverag
            Sequenc
            ##Assem
FEATURES
     source
```


Entrez Programming Utilities Help [Internet].

< Prev Next >

Show details

Contents

Search this book

E-utilities Quick Start

Eric Sayers, PhD.

Author Information and Affiliations

Created: December 12, 2008; Last Update: October 24, 2018.

Estimated reading time: 10 minutes

Release Notes

Go to:

Please see our Release Notes for details on recent changes and updates.

Announcement

Go to:

On December 1, 2018, NCBI will begin enforcing the use of new API keys for E-utility calls. Please see Chapter 2 for more details about this important change.

Introduction

Go to:

This chapter provides a brief overview of basic E-utility functions along with examples of URL calls. Please see Chapter 2 for a general introduction to these utilities and Chapter 4 for a detailed discussion of syntax and parameters.

Examples include live URLs that provide sample outputs.

All E-utility calls share the same base URL:

```
https://eutils.ncbi.nlm.nih.gov/entrez/eutils/
```

Searching a Database

Go to:

Basic Searching


```
PAVEALYR

DVYIRLPI
LRSRSSTF

RWRRKNGI
YYRIRKVK
PFSYHSRY
YNIRITMY

aaga
tccg
gttg
gtga
aaag
atag
acct
agaa
tgga
gcta
ggtt
gatc
cctt
tcaa
aaga
tgcc
ttat
acat
ggcc
tatt
agtg
aggt
ctcc
ttct
catt
tgac
agcg
ggct
acga
```

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk '$1=="AC"{print $2}' U31362.1.gb |tr -d ';'
U31362
```

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk '$1=="DE"{$1="";print $0}' U31362.1.gb
 Human immunodeficiency virus type 1 isolate IND7 envelope glycoprotein
 gp120 (env) mRNA, partial cds.
```

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk '/SQ/{get
line;while(i=1){print $0;getline; if ($1== "//"){break} } }' U31362.1.gb | tr -d '0-9'
     atgggagtga gggggatact gaggaattat caacaatggt ggatatgggg catcttaggc
     ttttggatgt taatgatttg taatgtggta ggaaacttgt gggtcacagt ctattatggg
     gtacctgtgt gggaagaagc aaaaactact ctattctgtg catcagatgc taaagcatat
     gagacagaag tgcataatgt ctgggctaca catgcctgtg tacccacaga ccccaaccca
     caagaaatat ttttggaaaa tgtaacagaa aattttaaca tgaggaaaaa tgacatggtg
     aatcagatgc atgaggatgt aatcagttta tgggatcaaa gcctaaagcc ctgtgtaaag
     ttgaccccac tctgtgtcac tttagaatgt agacaggtta atgttacca
     caggttaacg ctaccagtaa tggagaagaa ataaaaaata taaaaaatt
     tcaaccacag aaataagaga taggaagcag acagcgtatc gacttttt
     ctagtaccac ttgataacaa gaatgggagc aactctagta agtatatat
     aatacctcag ccataacaca agcctgtcca aaggtcactt ttgatccaa
     tattgtactc cagctggtta tgcgattcta aagtgtaatg ataagacat
     ggaccatgcc ataatgttag cacagtacaa tgtacacatg gaattaag
     actcaactac tgttaaatgg tagcctagca gaagaagaga taataatta
     ctgacagaca atgtcaaaac aataatagta catcttaatc aatctgtag
     acaagaccca acaataatac aagaaaaagt ataaggatag gaccaggac
     gcaacaggag acataatagg ggacataaga caagcacatt gtaacatta
     tggaatgaaa ctttacaaag ggtaagaaaa aaattagcag aacacttcc
     ataaatttta catcatcctc aggaggggac ctagaaatta caacacata
     agaggagaat ttttctattg taatcaatca ggcctgttta atggtacat
     ggtacaaaag gtaattcaag ctcagtcatc acaatcccat gcagaataa
     aacatgtggc agggggtagg acgagcaatg tatgcccctc ccattgaag
     tgtaaatcaa atatcacagg actactattg gtacgtgatg gaggactag
     gacacagaga cagagacatt cagacctgga ggaggagata tgagggacaa ttggagaagt
     gaattatata aatataaagt ggtaaaaatt aagccattgg gaatagcacc cactacagca
     aaaaggagag tggtggagag agaa
```

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk '/transla
tion/{while(i=1){print $0;getline; if ($1=="XX"){break} } }' U31362.1.gb|awk '{print
$2}'|tr -d '/translation="'
MGVRGILRNYQQWWIWGILGFWMLMICNVVGNLWVTVYYGVPVWE
EAKTTLFCASDAKAYETEVHNVWATHACVPTDPNPQEIFLENVTENFNMRKNDMVNQMH
EDVISLWDQSLKPCVKLTPLCVTLECRQVNVTSNGTQVNATSNGEEIKNIKNCSFNSTT
EIRDRKQTAYRLFYRLDLVPLDNKNGSNSSKYILINCNTSAITQACPKVTFDPIPIHYC
TPAGYAILKCNDKTFNGTGPCHNVSTVQCTHGIKPVVSTQLLLNGSLAEEEIIIRSENL
TDNVKTIIVHLNQSVEIVCTRPNNNTRKSIRIGPGQTFYATGDIIGDIRQAHCNISEAK
WNETLQRVRKKLAEHFPNKTINFTSSSGGDLEITTHSFNCRGEFFYCNQSGLFNGTYMH
NGTKGNSSSVITIPCRIKQIINMWQGVGRAMYAPPIEGNITCKSNITGLLLVRDGGLGP
SNDTETETFRPGGGDMRDNWRSELYKYKVVKIKPLGIAPTTAKRRVVERE
```

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk '$1=="AC"{print $2}' U31362.1.gb |tr -d ';'
U31362
```

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk '$1=="DE"{$1="";print $0}' U31362.1.gb
 Human immunodeficiency virus type 1 isolate IND7 envelope glycoprotein
 gp120 (env) mRNA, partial cds.
```

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk '/SQ/{get
line;while(i=1){print $0;getline; if ($1== "//"){break} } }' U31362.1.gb | tr -d '0-9'
     atgggagtga gggggatact gaggaattat caacaatggt ggatatgggg catcttaggc
     ttttggatgt taatgatttg taatgtggta ggaaacttgt gggtcacagt ctattatggg
     gtacctgtgt gggaagaagc aaaaactact ctattctgtg catcagatgc taaagcatat
     gagacagaag tgcataatgt ctgggctaca catgcctgtg tacccacaga ccccaaccca
     caagaaatat ttttggaaaa tgtaacagaa aattttaaca tgaggaaaaa tgacatggtg
     aatcagatgc atgaggatgt aatcagttta tgggatcaaa gcctaaagcc ctgtgtaaag
     ttgaccccac tctgtgtcac tttagaatgt agacaggtta atgttacca
     caggttaacg ctaccagtaa tggagaagaa ataaaaaata taaaaaatt
     tcaaccacag aaataagaga taggaagcag acagcgtatc gacttttt
     ctagtaccac ttgataacaa gaatgggagc aactctagta agtatatat
     aatacctcag ccataacaca agcctgtcca aaggtcactt ttgatccaa
```

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk '/transla
tion/{while(i=1){print $0;getline; if ($1=="XX"){break} } }' U31362.1.gb|awk '{print
$2}'|tr -d '/translation="'
```

```
#Acessionnumber
awk '$1=="AC"{print $2}' U31362.1.gb |tr -d ':'
#Gene_name
awk '$1=="DE"{$1="";print $0}' U31362.1.gb
awk 'BEGIN{ORS=" "} $1=="DE"{$1="";print $0}' U31362.1.gb
awk '$1=="DE"{print substr($0,6)}' U31362.1.gb
#Test1 SQ
awk '/SQ/{getline;while(i=1) {print $1,$2,$3,$4,$5,$6;getline; if ($1=="//"){break}}} END{gsub(" ","1524",$0)}' U31362.1.gb
#SQ
awk '/SQ/{getline;while(i=1){print $0;getline; if ($1=="//"){break} } }' U31362.1.gb | tr -d '0-9'
#Translation
awk '/translation/{while(i=1){print $0;getline; if ($1=="XX"){break} } }' U31362.1.gb|awk '{print $2}'|tr -d '/translation="'
```

With awk 1 command can do all?

```
awk '
$1=="AC"{print"\n";sub(";","",$0);print $2};
$1=="DE"{ORS=" ";print substr($0,6)}
/product/ {print"\n";ORS="\n";gsub("\"","",$0);$1="";print
substr($0,11)}
/translation/{gsub("/translation=","",$0);gsub("\"","",$0);
while(i=1){$1="";print $0;getline; if ($1=="XX"){break} } }
/SQ/{getline;while(i=1){$NF="";print $0;getline; if
($1=="//"){break} } }
' U31362.1.gb
```

```
U31362
Human immunodeficiency virus type 1 isolate IND7 envelope glycoprotein gp120 (env) mRNA, partial cds.
 envelope glycoprotein gp120
 MGVRGILRNYQQWWIWGILGFWMLMICNVVGNLWVTVYYGVPVWE
 EAKTTLFCASDAKAYETEVHNVWATHACVPTDPNPQEIFLENVTENFNMRKNDMVNQMH
 EDVISLWDQSLKPCVKLTPLCVTLECRQVNVTSNGTQVNATSNGEEIKNIKNCSFNSTT
 EIRDRKQTAYRLFYRLDLVPLDNKNGSNSSKYILINCNTSAITQACPKVTFDPIPIHYC
 TPAGYAILKCNDKTFNGTGPCHNVSTVQCTHGIKPVVSTQLLLNGSLAEEEIIIRSENL
 TDNVKTIIVHLNQSVEIVCTRPNNNTRKSIRIGPGQTFYATGDIIGDIRQAHCNISEAK
 WNETLQRVRKKLAEHFPNKTINFTSSSGGDLEITTHSFNCRGEFFYCNQSGLFNGTYMH
 NGTKGNSSSVITIPCRIKQIINMWQGVGRAMYAPPIEGNITCKSNITGLLLVRDGGLGP
 SNDTETETFRPGGGDMRDNWRSELYKYKVVKIKPLGIAPTTAKRRVVERE"
atgggagtga gggggatact gaggaattat caacaatggt ggatatgggg catcttaggc
ttttggatgt taatgatttg taatgtggta ggaaacttgt gggtcacagt ctattatggg
gtacctgtgt gggaagaagc aaaaactact ctattctgtg catcagatgc taaagcatat
gagacagaag tgcataatgt ctgggctaca catgcctgtg tacccacaga ccccaaccca
caagaaatat ttttggaaaa tgtaacagaa aattttaaca tgaggaaaaa tgacatggtg
aatcagatgc atgaggatgt aatcagttta tgggatcaaa gcctaaagcc ctgtgtaaag
ttgaccccac tctgtgtcac tttagaatgt agacaggtta atgttaccag taacggtaca
caggttaacg ctaccagtaa tggagaagaa ataaaaaata taaaaaattg ctctttcaat
tcaaccacag aaataagaga taggaagcag acagcgtatc gacttttta tagacttgat
ctagtaccac ttgataacaa gaatgggagc aactctagta agtatatatt aataaattgt
aatacctcag ccataacaca agcctgtcca aaggtcactt ttgatccaat tcctattcat
tattgtactc cagctggtta tgcgattcta aagtgtaatg ataagacatt caatgggaca
ggaccatgcc ataatgttag cacagtacaa tgtacacatg gaattaagcc agtagtatca
actcaactac tgttaaatgg tagcctagca gaagaagaga taataattag atctgaaaat
ctgacagaca atgtcaaaac aataatagta catcttaatc aatctgtaga aattgtgtgt
acaagaccca acaataatac aagaaaaagt ataaggatag gaccaggaca aacattctat
gcaacaggag acataatagg ggacataaga caagcacatt gtaacattag tgaagctaaa
tggaatgaaa ctttacaaag ggtaagaaaa aaattagcag aacacttccc taataaaaca
ataaatttta catcatcctc aggaggggac ctagaaatta caacacatag ctttaattgt
agaggagaat ttttctattg taatcaatca ggcctgttta atggtacata catgcataat
ggtacaaaag gtaattcaag ctcagtcatc acaatcccat gcagaataaa gcaaattata
aacatgtggc agggggtagg acgagcaatg tatgcccctc ccattgaagg aaacataaca
tgtaaatcaa atatcacagg actactattg gtacgtgatg gaggactagg accatcaaat
gacacagaga cagagacatt cagacctgga ggaggagata tgagggacaa ttggagaagt
gaattatata aatataaagt ggtaaaaatt aagccattgg gaatagcacc cactacagca
aaaaggagag tggtggagag agaa
```

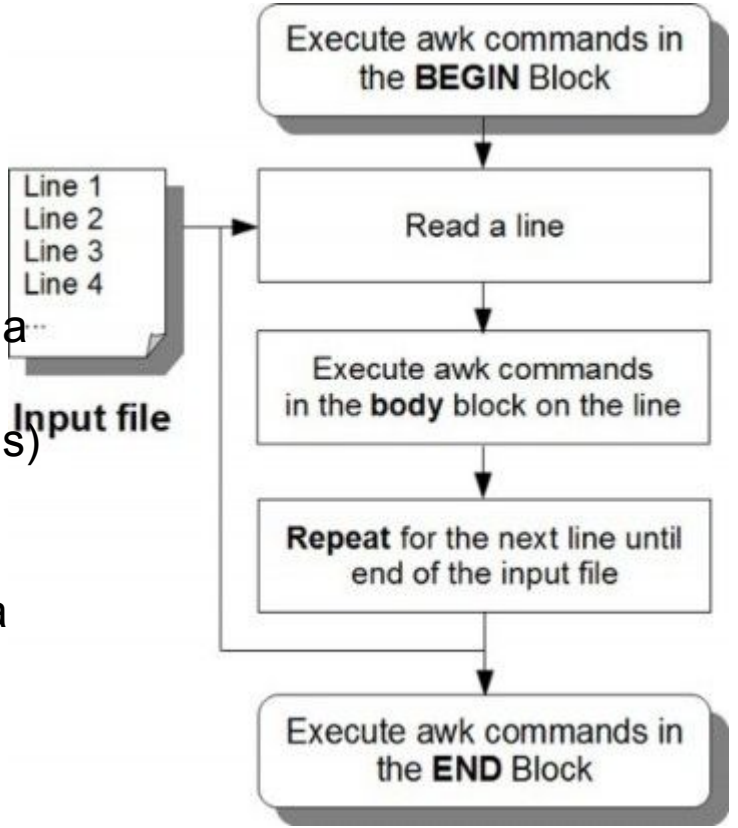
AWK cheat sheet

Basic learning awk

11.1 Introduction about awk

awk operates on a simple principle:
Line-by-Line Processing.

1. **Read:** It reads a file one line (record) at a time.
2. **Split:** It splits the line into fields (columns) based on whitespace (default) or a specific delimiter.
3. **Evaluate:** It checks if the line matches a specific pattern/condition.
4. **Execute:** If the pattern matches, it performs the specified action.



https://vds-admin.ru/sed-and-awk-101-hacks/chapter-8-awk-syntax-and-basic-commands

Why we use awk in bioinformatic?

Column number	Title	Definition
1	chrom	Chromosome (e.g. chr3, chrY, chr2_random) or *scaffold* (e.g. scaffold10671) name
2	chromStart	Start coordinate on the chromosome or scaffold for the sequence considered (the first base on the chromosome is numbered 0 i.e. the number is zero-based)
3	chromEnd	End coordinate on the chromosome or scaffold for the sequence considered. This position is non-inclusive, unlike chromStart (the first base on the chromosome is numbered 1 i.e. the number is one-based).
4	name	Name of the line in the BED file
5	score	Score between 0 and 1000
6	strand	DNA strand orientation (positive ["+"] or negative ["-"] or "." if no strand)
7	thickStart	Starting coordinate from which the annotation is displayed in a thicker way on a graphical representation (e.g.: the start codon of a gene)
8	thickEnd	End coordinates from which the annotation is no longer displayed in a thicker way on a graphical representation (e.g.: the stop codon of a gene)
9	itemRgb	RGB value in the form R, G, B (e.g. 255,0,0) determining the display color of the annotation contained in the BED file
10	blockCount	Number of blocks (e.g. exons) on the line of the BED file
11	blockSizes	List of values separated by commas corresponding to the size of the blocks (the number of values must correspond to that of the "blockCount")
12	blockStarts	List of values separated by commas corresponding to the starting coordinates of the blocks, coordinates calculated relative to those present in the chromStart column (the number of values must correspond to that of the "blockCount")

https://en.wikipedia.org/wiki/BED_(file_format)

Field

```
browser position      :127471196-127495720
browser hide all
track name="ItemR     mo" description="Item RGB demonstration" visibility=2 itemRgb="On"
chr7    127471196    127472363    Pos1    0    +    127471196    127472363    255,0,0
chr7    127472363    127473530    Pos2    0    +    127472363    127473530    255,0,0
chr7    127473530    127474697    Pos3    0    +    127473530    127474697    255,0,0
chr7    127474697    127475864    Pos4    0    +    127474697    127475864    255,0,0
chr7    127475864    127477031    Neg1    0    -    127475864    127477031    0,0,255
chr7    127477031    127478198    Neg2    0    -    127477031    127478198    0,0,255
chr7    127478198    127479365    Neg3    0    -    127478198    127479365    0,0,255
chr7    127479365    127480532    Pos5    0    +    127479365    127480532    255,0,0
chr7    12748053     127481699    Neg4    0    -    127480532    127481699    0,0,255
```

Why we use awk in bioinformatic?

Column number	Title	Definition
1	chrom	Chromosome (e.g. chr3, chrY, chr2_random) or *scaffold* (e.g. scaffold10671) name
2	chromStart	Start coordinate on the chromosome or scaffold for the sequence considered (the first base on the chromosome is numbered 0 i.e. the number is zero-based)
3	chromEnd	End coordinate on the chromosome or scaffold for the sequence considered. This position is non-inclusive, unlike chromStart (the first base on the chromosome is numbered 1 i.e. the number is one-based).
4	name	Name of the line in the BED file
5	score	Score between 0 and 1000
6	strand	DNA strand orientation (positive ["+"] or negative ["-"] or "." if no strand)
7	thickStart	Starting coordinate from which the annotation is displayed in a thicker way on a graphical representation (e.g.: the start codon of a gene)
8	thickEnd	End coordinates from which the annotation is no longer displayed in a thicker way on a graphical representation (e.g.: the stop codon of a gene)
9	itemRgb	RGB value in the form R, G, B (e.g. 255,0,0) determining the display color of the annotation contained in the BED file
10	blockCount	Number of blocks (e.g. exons) on the line of the BED file
11	blockSizes	List of values separated by commas corresponding to the size of the blocks (the number of values must correspond to that of the "blockCount")
12	blockStarts	List of values separated by commas corresponding to the starting coordinates of the blocks, coordinates calculated relative to those present in the chromStart column (the number of values must correspond to that of the "blockCount")

https://en.wikipedia.org/wiki/BED_(file_format)

Record

```
browser position chr7:127471196-127495720
browser hide all
track name="ItemRGBDemo" description="Item RGB demonstration" visibility=2 itemRgb="On"
chr7    127471196    127472363    Pos1    0    +    127471196    127472363    255,0,0
chr7    127472363    127473530    Pos2    0    +    127472363    127473530    255,0,0
chr7    127473530    127474697    Pos3    0    +    127473530    127474697    255,0,0
chr7    127474697    127475864    Pos4    0    +    127474697    127475864    255,0,0
chr7    127475864    127477031    Neg1    0    -    127475864    127477031    0,0,255
chr7    127477031    127478198    Neg2    0    -    127477031    127478198    0,0,255
chr7    127478198    127479365    Neg3    0    -    127478198    127479365    0,0,255
chr7    127479365    127480532    Pos5    0    +    127479365    127480532    255,0,0
```

Preparation: The Dataset

echo -e “ID Name Dept Salary \n 101 John Sales 5000 \n 102 Alice Eng 9000 \n 103 Bob Sales 5200 \n 104 Carol HR 4500 \n 105 Dave Eng 8800 \n 106 Eve HR 4700” > data.txt

Plaintext

```
ID Name Dept Salary
101 John Sales 5000
102 Alice Eng 9000
103 Bob Sales 5200
104 Carol HR 4500
105 Dave Eng 8800
106 Eve HR 4700
```

11.1 Field delimiter

“:” is a field separator

awk -F<value> **#Default is “ “ #Set input field separator**

```
(base) huyha@dummycomputer:~$ echo "Huy:M:150:18:Hochiminh" | awk -F: '{print $1}'
Huy
(base) huyha@dummycomputer:~$ echo "Huy:M:150:18:Hochiminh" | awk -F: '{print $1,$3}'
Huy 150
(base) huyha@dummycomputer:~$ echo "Huy:M:150:18:Hochiminh" | awk -F: '{print $0}'
Huy:M:150:18:Hochiminh
```

awk -f <filename> **#Reading commands to activate in a file**

```
echo '{print $1} END{print "this is the end of process"}' >awk1
awk -f awk1 Test1.txt
```

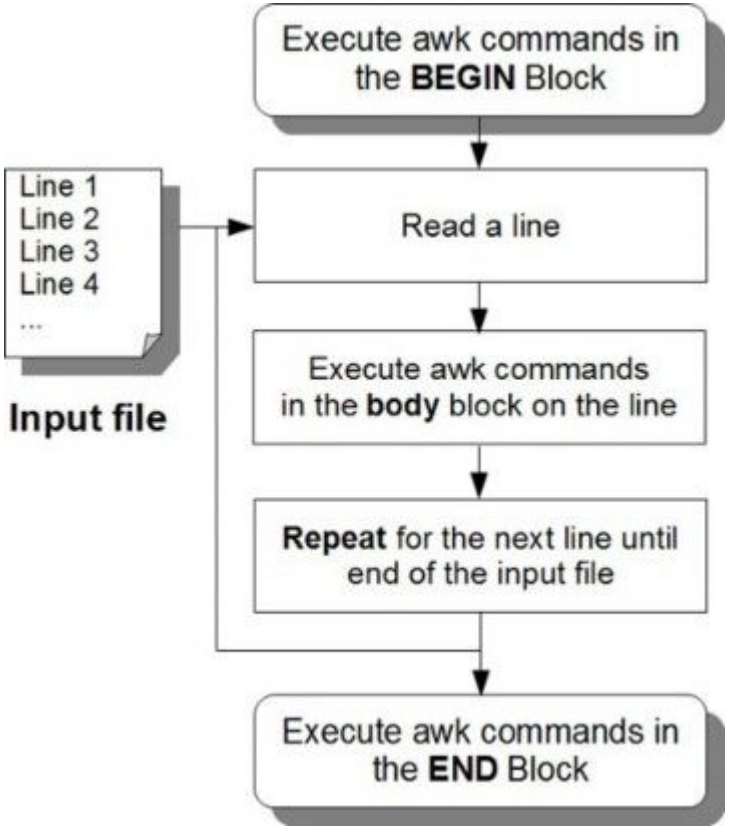
11.1 Awk structure

awk -option

‘**BEGIN**{action beginblock}
/**pattern**/ {action bodyblock}

END{action endblock}’

input_file



11.2 Regular Expressions (Regex)

baschscript > Test1.txt

```
1  Name sex city age
2  Linh female Haiphong 21
3  Nam male Hochiminh 55
4  Mai female Hochiminh 26
5  Vinh male Hanoi 12
6  Xuan female Binhdinh 53
```

Test1.txt

Regular expression	
/regex/	Line matches
!/regex/	Line not matches
$1 ~ /regex/	Field matches
$1 !~ /regex/	Field not matches

11.2 Regular Expressions (Regex)

baschscript > Test1.txt

```
1  Name sex city age
2  Linh female Haiphong 21
3  Nam male Hochiminh 55
4  Mai female Hochiminh 26
5  Vinh male Hanoi 12
6  Xuan female Binhdinh 53
```

Test1.txt

awk '/female/ {print $0}' Test1.txt

```
Linh female Haiphong 21
Mai female Hochiminh 26
Xuan female Binhdinh 53
Truc female Haiphong 21
```

awk ' $2 == male {print $0}' Test1.txt

```
Nam male Hochiminh 55
Vinh male Hanoi 12
```

awk '/chi/ {print $0}' Test1.txt

```
Nam male Hochiminh 55
Mai female Hochiminh 26
```

Advanced Field Regex (Anchors)

Example: Find employees whose **Name** starts with "D".

Bash

```
awk '$2 ~ /^D/' data.txt
```

Output:

Plaintext

```
105 Dave Eng 8800
```

Example: Find salaries that end in "00".

Bash

```
awk '$4 ~ /00$/' data.txt
```

When using Field Matching, you can be even more precise using **Anchors**:

- ^ = Starts with
- $ = Ends with

Summary Table

Type	Syntax	Meaning
Line Match	`/regex/`	Is the pattern anywhere in the full line?
Field Match	`$N ~ /regex/`	Is the pattern inside column N?
Negative Match	`$N !~ /regex/`	Is the pattern MISSING from column N?

11.2 AWK Variable

```
# Access 'my_var'
inside the script

my_var="hello"

awk -v var="$my_var"
 'BEGIN { print var }'

hello
```

Build-in variables

`$0`	Whole line
`$1, $2...$NF`	First, second... last field
`NR`	Number of Records
`NF`	Number of Fields
`OFS`	Output Field Separator (default " ")
`FS`	input Field Separator (default " ")
`ORS`	Output Record Separator (default "\n")
`RS`	input Record Separator (default "\n")
`FILENAME`	Name of the file

Expressions

`$1 == "root"`	First field equals root
`{print $(NF-1)}`	Second last field
`NR!=1{print $0}`	From 2th record
`NR > 3`	From 4th record
`NR == 1`	First record
`END{print NR}`	Total records
`BEGIN{print OFMT}`	Output format
`{print NR, $0}`	Line number
`{print NR " " $0}`	Line number (tab)
`{$1 = NR; print}`	Replace 1th field with line number
`$NF > 4`	Last field > 4
`NR % 2 == 0`	Even records
`NR==10, NR==20`	Records 10 to 20
`BEGIN{print ARGC}`	Total arguments
`ORS=NR%5?",":"\n"`	Concatenate records

AWK Standard Variables (1)

FILENAME: is an AWK **built-in variable** that holds the name of the **current file being processed** by the AWK script.

Input: `awk 'END {print FILENAME}' myfile.txt`

Output: `myfile.txt`

FS: is an AWK **built-in variable** that represents the input **field separator**.

Input:

```
cat myfile.txt
```

```
,V1
```

```
R2,V2
```

```
awk  -v FS=',' '{print $1}' myfile.txt
```

Output:

```
R2
```

AWK Standard Variables (2)

NF: stands for "**Number of Fields**." Field is a part of the input line that is separated by a delimiter, which is **whitespace by default**.

Code:
```
echo -e "One Two\nOne Two Three\nOne Two Three Four" | awk 'NF > 2'
```

Input:

```
One Two

One Two Three

One Two Three Four
```

Output:

```
One Two Three

One Two Three Four
```

NR: stands for "**Number of Records**." Record is typically **a line of input**.

Code:
```
echo -e "One Two\nOne Two Three\nOne Two Three Four" | awk 'NR < 3'
```

Input:

```
One Two

One Two Three

One Two Three Four
```

Output:

```
One Two

One Two Three
```

AWK Standard Variables (3)

FNR: is similar to NR, but relative to the current file. When AWK processes multiple files, FNR represents the current line number within the current file being processed.

OFS: stands for "**Output Field Separator**." It determines the character or string used to separate fields when printing the output. The **default value is a whitespace**.

RS: stands for the "**Record Separator**" for input records. It specifies the character or string that separates records (lines) in the input. The **default value is a newline**.

ORS: stands for "**Output Record Separator**." It defines the character or string used to separate output records (lines) when printing the result. The **default value is a newline**.

$0: represents the entire input record (the entire line of input).

$n: represents the nth field in the current input record (line), where the fields are separated by the field separator **FS**.

11.4 Operator

Addition and Assignment: +=

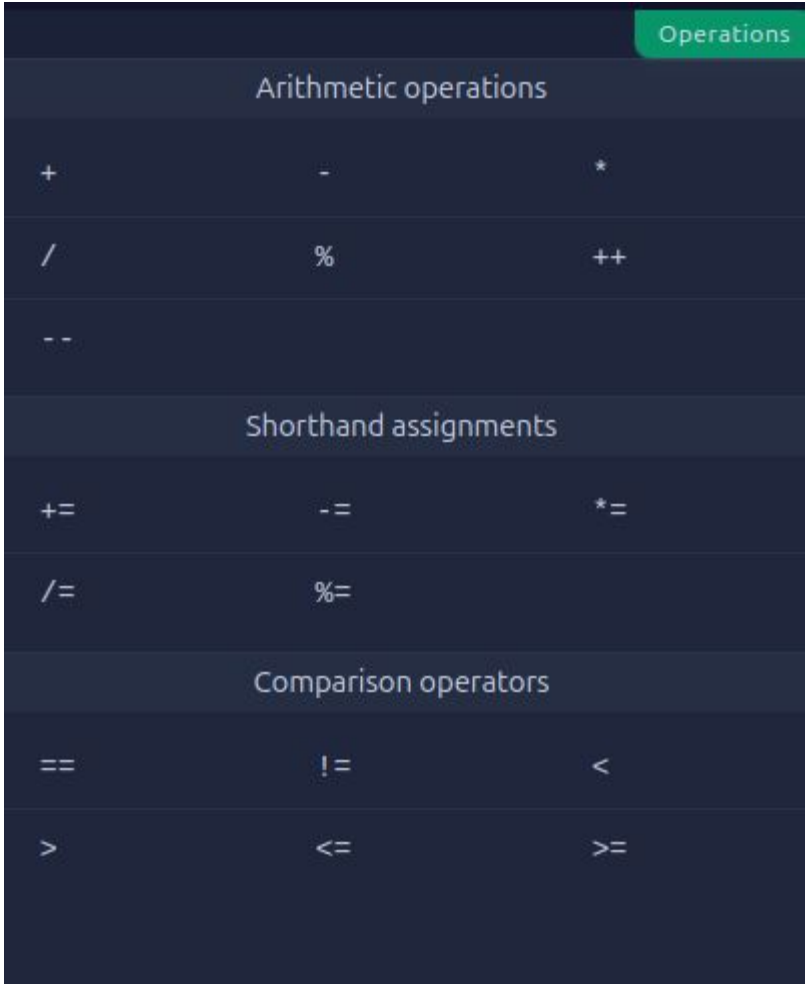
```
num = 5;

num += 3; // Equivalent to 'num = num + 3'

# Now 'num' is 8
```

Subtraction and Assignment: -=

```
num = 10;

num -= 4; // Equivalent to 'num = num - 4'

# Now 'num' is 6
```

Multiplication and Assignment: *=

```
num = 3;

num *= 5; // Equivalent to 'num = num * 5'

# Now 'num' is 15
```



getline #go down 1 row

11.3 Functions

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk 'sub(" ", ":") {print}' Test1.txt
Name:sex city age
Linh:female Haiphong 21
Nam:male Hochiminh 55
Mai:female Hochiminh 26
Vinh:male Hanoi 12
Xuan:female Binhdinh 53
Truc:female Haiphong 21
```

sub(/a/,"b",$0) #change first "a" to b in row

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk 'gsub(" ", ":") { print $0}' Test1.t
Name:sex:city:age
Linh:female:Haiphong:21
Nam:male:Hochiminh:55
Mai:female:Hochiminh:26
Vinh:male:Hanoi:12
Xuan:female:Binhdinh:53:
Truc:female:Haiphong:21
```

gsub(/a/,"b",$0) # change every "a" to b in row

```
awk 'BEGIN{print substr("FirstName",6)}'
Name
```

substr("Hello",3) #return llo

11.4 Increment and decrement operator examples

Pre-increment:

```
num = 3;

result = ++num; // 'num' is incremented
to 3 before its value is assigned to
'result'

print num, result

# Now 'num' is 4, and 'result' is also 4
```

Post-increment:

```
num = 3;

result = num++; // 'num' is used as 3 in
the expression, then it is incremented to
4

print num, result

# Now 'num' is 4, and 'result' is 3
(previous value of 'num')
```

```
awk 'BEGIN {num = 3; preincr = ++num; print num, preincr;
            num = 3; postincr = num++; print num, postincr}'

4 4
4 3
```

11.5 Awk Condition

If statement **#if(condition) {command block} else {command block}**

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk '{if (NR > 1)  print NR,$0}' Test1.txt
2 Linh female Haiphong 21
3 Nam male Hochiminh 55
4 Mai female Hochiminh 26
5 Vinh male Hanoi 12
6 Xuan female Binhdinh 53
7 Truc female Haiphong 21
```

```
awk 'END{
        if  (NF % 2 == 0)
            {print "even"}
        else
            {print"odd"}
        {print NF}
        }' Test1.txt
```

```
awk 'END{if(NF % 2 == 0) {print "even"} else {print"odd"} {print NF}}' Test1.txt
```

```
even
4
```

11.6 Awk loop

#For (condition; variable; Change) {action block}

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk ' BEGIN{ for ( i=1; i<5; i++ ) { print i }}'
1
2
3
4
```

#While (condition) {action block}

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ awk ' BEGIN{ while ( i<10 ){ i+=2; print i}}'
2
4
6
8
10
```

Next statement, Break statement

```
# next statement
{
    if ($1 == "skip") {
        next;  # Skip processing this record and move to the next one
    };
    print "Processing:", $1;
}
```

```
# break statement
BEGIN {
    for (i = 1; i <= 10; i++) {
        if (i == 6) {
            break;  # Exit the loop when 'i' becomes 6
        };
        print "Iteration:", i;
    }
}
```

Assignment

Test your knowledge with these three tasks.

Task 1: Basic Extraction Write a command to print **only the Names** (Column 2) of employees in the "HR" department.

Task 2: Formatting Write a command that prints the row number, then the Name, then the Department in this format: `Row 1: John is in Sales`

Task 3: Analytics Calculate the **average** salary of all employees. *Hint: You need a sum variable, and you can use `NR` (minus the header row) to divide.*

```
ID Name Dept Salary
101 John Sales 5000
102 Alice Eng 9000
103 Bob Sales 5200
104 Carol HR 4500
105 Dave Eng 8800
106 Eve HR 4700
```

Awk homework 2

```
Down file using below command
curl -o "./U31362.1.gb" https://www.ebi.ac.uk/ena/browser/api/embl/U31362.1?download=true
cat U31362.1.gb
```

1. Find accession number
2. Find product name
3. Find translation sequence

4. (optional) write script to automatic down file input = list of accession numbers

```
CDS             1..>1524
                /codon_start=1
                /gene="env"
                /product="envelope glycoprotein gp120"
                /db_xref="GOA:Q72858"
                /db_xref="InterPro:IPR000777"
                /db_xref="InterPro:IPR036377"
                /db_xref="UniProtKB/TrEMBL:Q72858"
                /protein_id="AAC55476.1"
                /translation="MGVRGILRNYQQWWIWGILGFWMLMICNVVGNLWVTVYYGVPVWE
                EAKTTLFCASDAKAYETEVHNVWATHACVPTDPNPQEIFLENVTENFNMRKNDMVNQMH
                EDVISLWDQSLKPCVKLTPLCVTLECRQVNVTSNGTQVNATSNGEEIKNIKNCSFNSTT
                EIRDRKQTAYRLFYRLDLVPLDNKNGSNSSKYILINCNTSAITQACPKVTFDPIPIHYC
                TPAGYAILKCNDKTFNGTGPCHNVSTVQCTHGIKPVVSTQLLLNGSLAEEEIIIRSENL
                TDNVKTIIVHLNQSVEIVCTRPNNNTRKSIRIGPGQTFYATGDIIGDIRQAHCNISEAK
                WNETLQRVRKKLAEHFPNKTINFTSSSGGDLEITTHSFNCRGEFFYCNQSGLFNGTYMH
                NGTKGNSSSVITIPCRIKQIINMWQGVGRAMYAPPIEGNITCKSNITGLLLVRDGGLGP
                SNDTETETFRPGGGDMRDNWRSELYKYKVVKIKPLGIAPTTAKRRVVERE"
```